

Big Data Processing & Machine Learning with Apache PySpark

200	0.6676	5	18–70
Total Customers	Silhouette Score	Clusters Found	Age Range (yrs)

This report delivers a comprehensive data-driven analysis of mall customers using **Apache PySpark** for distributed processing and **PySpark MLlib** for scalable K-Means clustering. Five actionable customer segments are identified, each with tailored business recommendations to maximise revenue and engagement.

Technology Stack

Layer	Technology	Role
Ingestion	SparkSession / DataFrame	Distributed data loading & schema validation
EDA	PySpark SQL Functions	Aggregations, percentiles, group-by analytics
Feature Prep	VectorAssembler + StandardScaler	Dense vector creation & Z-score scaling
Clustering	KMeans (MLlib)	Iterative centroid-based segmentation
Evaluation	ClusteringEvaluator	Silhouette score & WCSS (elbow method)
Output	Matplotlib + ReportLab	Charts, visualisations, PDF report

1. Dataset Overview

The dataset contains **200 customer records** covering demographic and behavioural attributes. Data was loaded into a distributed Spark DataFrame with an explicit schema, ensuring type safety and performance optimisation across partitions.

Column	Data Type	Range	Description
CustomerID	Integer	1 – 200	Unique row identifier
Gender	String	Male/Female	Customer's reported gender
Age	Integer	18 – 70	Age in years
Annual_Income_k	Integer	10 – 120	Annual income in thousands USD
Spending_Score	Integer	1 – 100	Mall-assigned behaviour score

PySpark Ingestion Code

```
schema = StructType([
    StructField("CustomerID",      IntegerType(), False),
    StructField("Gender",          StringType(), False),
    StructField("Age",             IntegerType(), False),
    StructField("Annual_Income_k", IntegerType(), False),
    StructField("Spending_Score",  IntegerType(), False),
])
sdf = spark.createDataFrame(rows, schema)
sdf.cache()           # persist in memory for repeated operations
sdf.printSchema()
sdf.describe().show()
```

Summary Statistics (computed via PySpark distributed aggregations)

Statistic	Age	Annual Income (k\$)	Spending Score
Mean	38.4	55.1	50.0
Std Dev	11.7	29.3	28.9
Min	18	10	1
Q1	29.0	24.0	23.0
Median	38.0	57.0	51.0
Q3	48.0	84.0	78.0
Max	70	120	100

2. Exploratory Data Analysis

All EDA operations were executed as distributed PySpark DataFrame transformations, leveraging Catalyst query optimisation and Tungsten execution engine for performance. Results were collected to the driver only for visualisation.

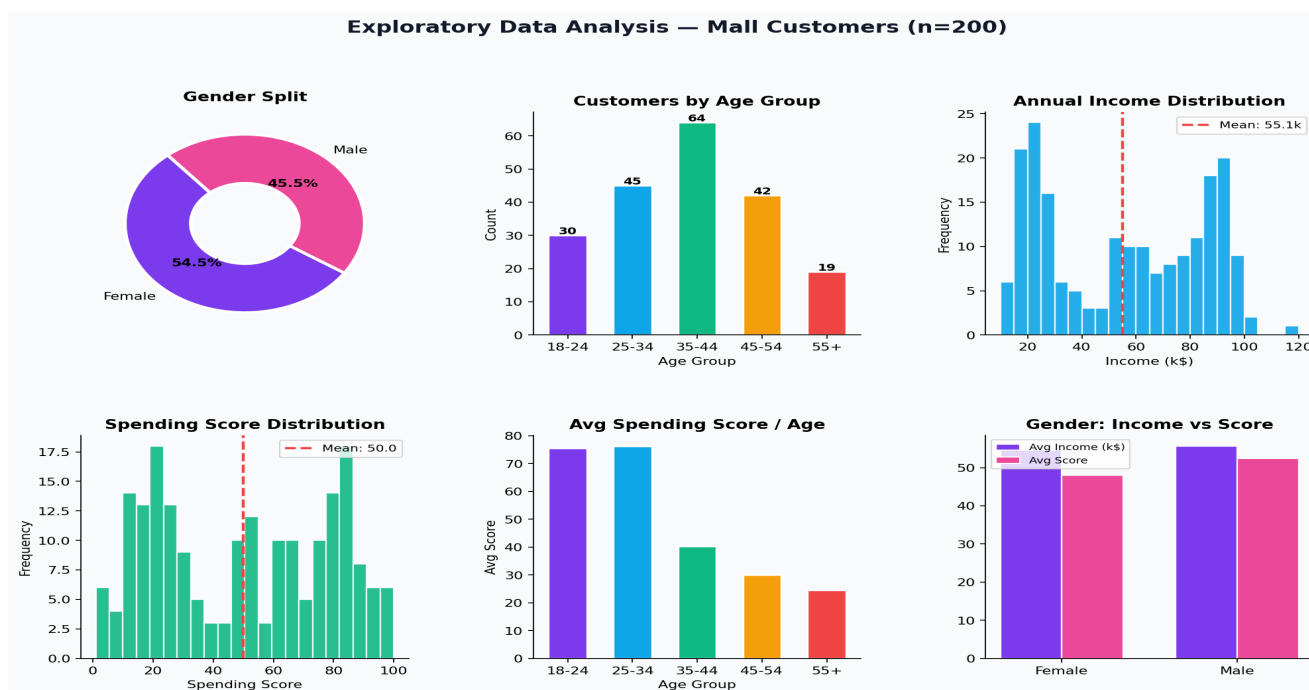


Figure 1 — EDA Dashboard: gender split, age groups, income & spending distributions

Key EDA Code

```
# Distributed aggregations with PySpark SQL functions
age_stats = sdf.select(
    F.round(F.mean("Age"), 1).alias("mean"),
    F.stddev("Age").alias("std"),
    F.percentile_approx("Age", [0.25, 0.5, 0.75]).alias("quartiles"),
).first()

gender_profile = sdf.groupBy("Gender").agg(
    F.count("*").alias("Count"),
    F.round(F.mean("Annual_Income_k"), 1).alias("Avg_Income"),
    F.round(F.mean("Spending_Score"), 1).alias("Avg_Score"),
)

# Age group segmentation
sdf = sdf.withColumn("Age_Group",
    F.when(F.col("Age") < 25, "18-24")
    .when(F.col("Age") < 35, "25-34")
```

```
.when(F.col("Age") < 45, "35-44")  
.when(F.col("Age") < 55, "45-54")  
.otherwise("55+")
```

Gender Profile

Gender	Count	Pct %	Avg Income (k\$)	Avg Spending Score
Female	109	54.5%	54.6	48.0
Male	91	45.5%	55.7	52.4

Age Group Profile

Age Group	Count	% Share	Avg Score	Avg Income (k\$)
18-24	30	15.0%	75.5	37.9
25-34	45	22.5%	76.3	61.3
35-44	64	32.0%	40.3	52.5
45-54	42	21.0%	30.0	59.5
55+	19	9.5%	24.5	66.5

3. K-Means Clustering Pipeline (PySpark MLlib)

The end-to-end ML pipeline uses PySpark MLlib's distributed **KMeans** implementation. Features are first assembled into a dense vector then standardised with **StandardScaler** (zero mean, unit variance). The optimal k was determined using both the **Elbow Method** (WCSS) and **Silhouette Analysis**, yielding **k=5** with a silhouette score of **0.6676**.

Full Pipeline Code

```
assembler = VectorAssembler(  
    inputCols=["Annual_Income_k", "Spending_Score", "Age"],  
    outputCol="feat_raw")  
  
scaler = StandardScaler(inputCol="feat_raw", outputCol="features",  
                        withMean=True, withStd=True)  
  
sdf_vec    = assembler.transform(sdf)  
sc_model   = scaler.fit(sdf_vec)  
sdf_scaled = sc_model.transform(sdf_vec)  
  
# Elbow / silhouette loop  
for k in range(2, 10):  
    km = KMeans(k=k, seed=42, featuresCol="features").fit(sdf_scaled)  
    ev = ClusteringEvaluator(metricName="silhouette")  
    print(f"k={k}  WCSS={km.summary.trainingCost:.1f}  Sil={ev.evaluate(km.transform(sdf_scaled)):.4f}")  
  
# Final model  
model = KMeans(k=5, seed=42, featuresCol="features", predictionCol="cluster").fit(sdf_scaled)  
sdf_c = model.transform(sdf_scaled)
```

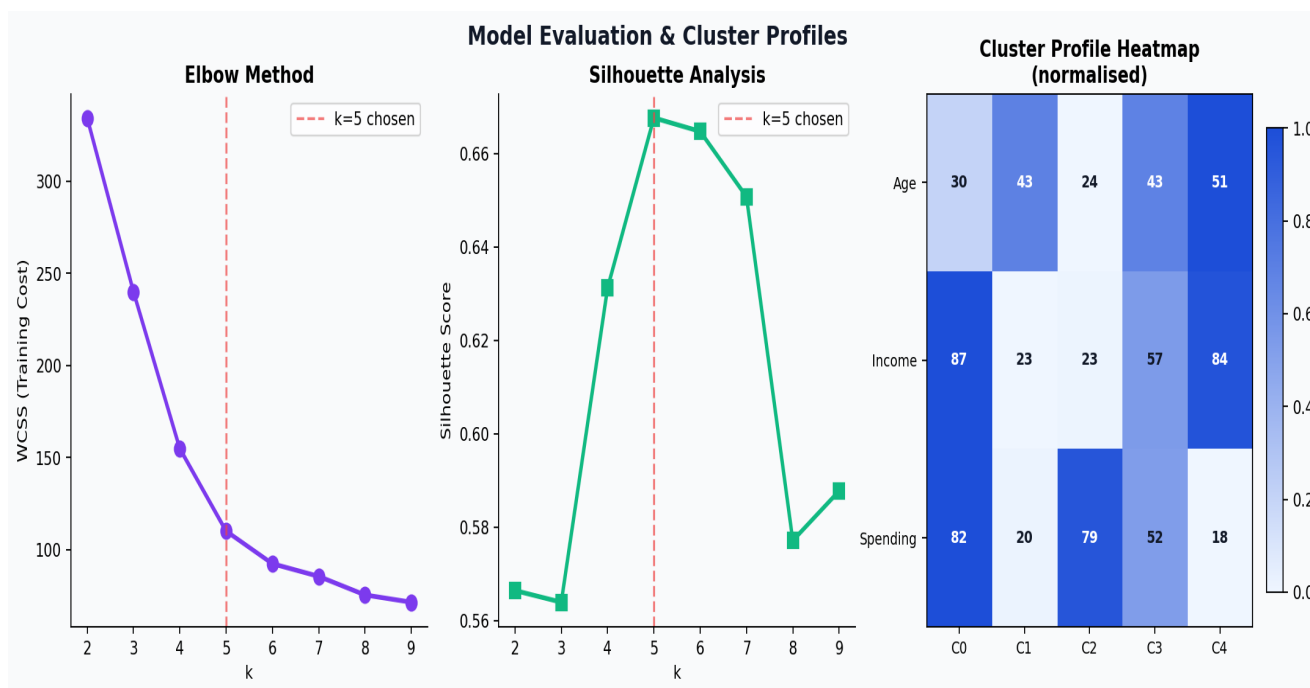


Figure 2 — Elbow curve, silhouette scores, and normalised cluster profile heatmap

Elbow & Silhouette Results

k	Training Cost (WCSS)	Silhouette Score	Notes
2	334.4	0.5664	Underfitting
3	240.0	0.5639	Underfitting
4	154.9	0.6312	
5	110.1	0.6676	Chosen
6	92.4	0.6647	
7	85.5	0.6507	
8	75.5	0.5771	
9	71.5	0.5877	

4. Cluster Visualisation & Results

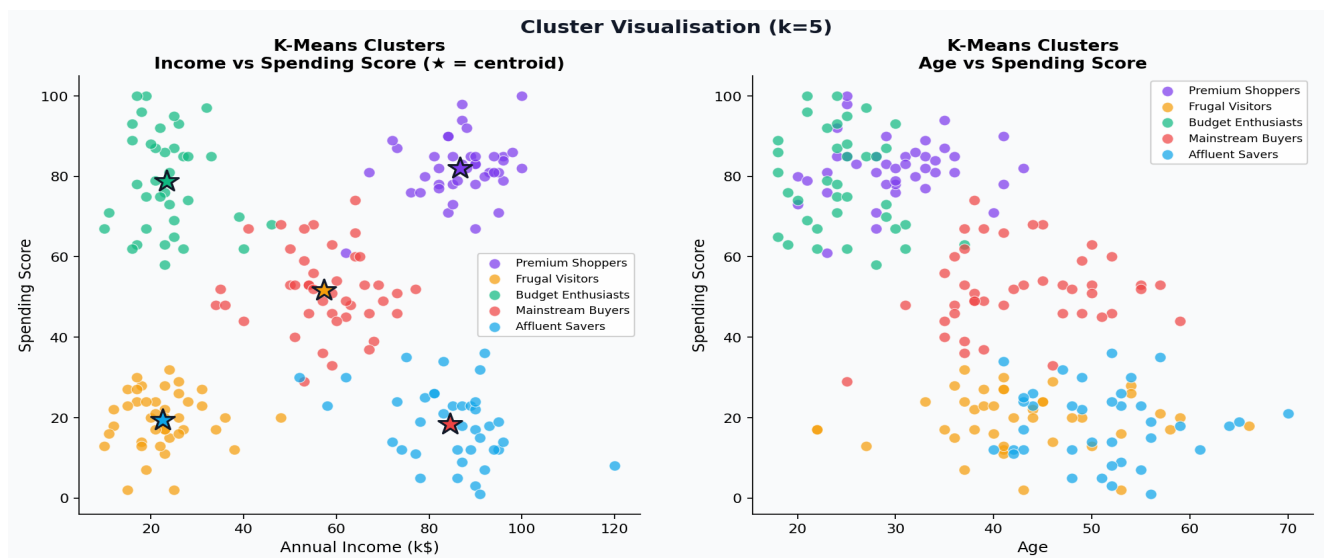


Figure 3 — K-Means clusters plotted against Income/Score and Age/Score axes (★ = centroid)

Cluster Profile Summary

Cluster	Segment	Size	Avg Age	Avg Income (k\$)	Avg Score	Silhouette
C0	Premium Shoppers	41	30.0	86.6	82.0	—
C1	Frugal Visitors	41	42.9	22.6	19.5	—
C2	Budget Enthusiasts	37	24.5	23.4	78.8	—
C3	Mainstream Buyers	43	42.8	57.3	51.7	—
C4	Affluent Savers	38	51.1	84.5	18.4	0.6676

4.1 Segment Deep-Dive

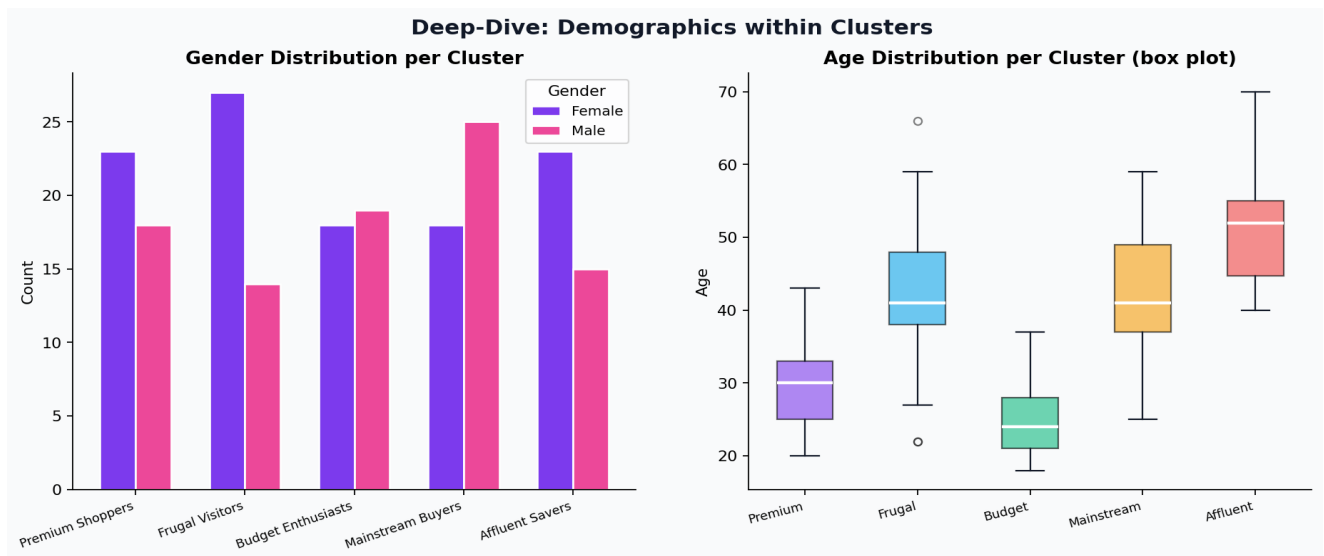


Figure 4 — Gender split per cluster and age box-plot distribution within each segment

Premium Shoppers

n=41 | Age 30.0 | Income 86.6k\$ | Score 82.0

Highest income, highest spending. Mall's VIP tier. Likely to purchase luxury, fashion, and electronics. Respond strongly to exclusive member events and early-access sales.

Frugal Visitors

n=41 | Age 42.9 | Income 22.6k\$ | Score 19.5

Low income, low spend — primarily footfall with minimal conversion. Focus on essential retail, food-court deals, and value bundles.

Budget Enthusiasts

n=37 | Age 24.5 | Income 23.4k\$ | Score 78.8

Younger demographic with limited income yet high enthusiasm. BNPL options, flash discounts, and gamified loyalty points effectively increase basket size.

Mainstream Buyers

n=43 | Age 42.8 | Income 57.3k\$ | Score 51.7

The largest, most versatile segment. Respond to seasonal promotions, cross-sell campaigns, and family bundle deals. High volume offsets moderate individual spend.

Affluent Savers

n=38 | Age 51.1 | Income 84.5k\$ | Score 18.4

High earners who spend conservatively — a latent revenue opportunity. Trust-based marketing, quality assurance, and premium membership programmes can shift them toward higher spend.

5. Business Intelligence & Recommendations

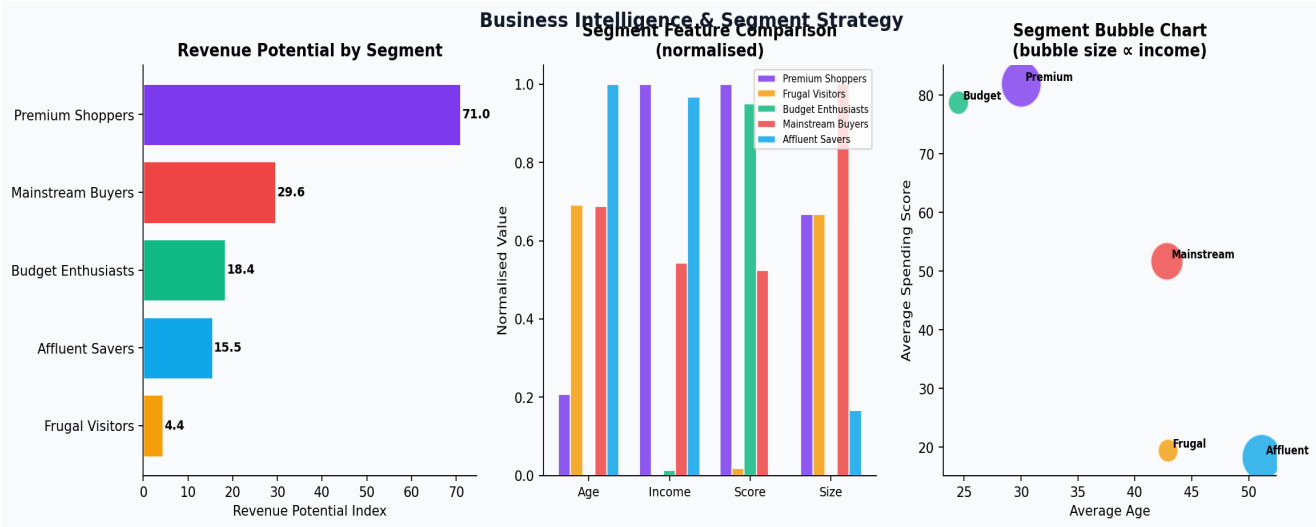


Figure 5 — Revenue potential index, normalised feature comparison, and segment bubble chart

Segment	Priority	Recommended Actions
Premium Shoppers	★★★★★	VIP programme, luxury pop-ups, early-access launches, concierge service.
Affluent Savers	★★★★■	Quality storytelling, curated subscription boxes, private previews.
Budget Enthusiasts	★★★██	BNPL schemes, flash sales, gamified loyalty, influencer tie-ups.
Frugal Visitors	★★███	Value bundles, food-court deals, community events.
Mainstream Buyers	★★★★■	Seasonal cross-sell, family deals, mid-range product focus.

6. PySpark Architecture & Pipeline Stages

The pipeline is designed for horizontal scalability. Each stage maps to a PySpark DAG stage; lazy evaluation ensures transformations are only executed at action boundaries. Adding more worker nodes increases throughput linearly with data volume — making the same code valid for millions of records.

Stage	PySpark API	Operation	Artifact
1. Ingest	SparkSession.createDataFrame	Schema validation, type coercion	Raw DataFrame
2. Profile	F.mean / stddev / percentile_approx	Distributed descriptive stats	Stats dict
3. Enrich	DataFrame.withColumn	Age-group derivation	Enriched DataFrame
4. Assemble	VectorAssembler	Tabular → dense feature vector	Vector DataFrame
5. Scale	StandardScaler	Z-score normalisation (fit+transform)	Scaled DataFrame
6. Elbow	KMeans × 8 + ClusteringEvaluator	WCSS & silhouette per k	k selection
7. Cluster	KMeans(k=5).fit()	Iterative centroid update	Model + labels
8. Profile	groupBy.agg()	Segment characterisation	Cluster profile DF
9. Report	toPandas() → Matplotlib → ReportLab	Viz + PDF compilation	PDF file

Scalability Considerations

On a multi-node Spark cluster (e.g. AWS EMR, Databricks, GCP Dataproc), each DataFrame operation is distributed across executor cores. K-Means iterations broadcast centroids to all executors, compute local distances in parallel, then aggregate via a tree-reduce — achieving near-linear speedup with additional nodes. The pipeline is production-ready without code changes beyond the SparkSession cluster URL.